Proyecto 2 - Reconocimiento con gramáticas libres de contexto

1. Gramática

La gramática es G = (V, Sigma, S, P), con:

- V = {S, VP, PP, NP, V, P, N, Det}
- Sigma = {a, beer, cake, cat, cooks, cuts, dog, drinks, eats, fork, he, in, juice, knife, meat, oven, she, soup, spoon, the, with}
- S = S

Producciones:

```
S   -> NP VP
VP  -> VP PP | V NP | cooks | drinks | eats | cuts
PP  -> P NP
NP  -> Det N | he | she
V   -> cooks | drinks | eats | cuts
P   -> in | with
N   -> cat | dog | beer | cake | juice | meat | soup
N   -> fork | knife | oven | spoon
Det -> a | the
```

Hay 30 producciones, 8 no terminales y 21 terminales. Todas las producciones tienen la forma A -> a o A -> B C (CNF).

La gramática genera oraciones formadas por un sintagma nominal seguido de un sintagma verbal. El sintagma verbal puede ser un verbo, un verbo con objeto o un sintagma verbal seguido de uno o más sintagmas preposicionales. Los sintagmas preposicionales se adjuntan al VP, no al NP. Por eso las frases de este lenguaje tienen una única derivación.

2. Conversión de la gramática a un PDA

Se usa la conversión estándar de una gramática libre de contexto a un autómata con pila no determinista.

El PDA es M = (Q, Sigma, Gamma, delta, q0, Z0, F), con:

- Q = {q0, q1, q2}
- Sigma = alfabeto de terminales de G
- Gamma = V union Sigma union {Z0}
- q0 = estado inicial
- Z0 = símbolo inicial de la pila
- F = {q2}

Transiciones:

```
delta(q0, epsilon, Z0) = {(q1, S Z0)}
delta(q1, epsilon, A)  = {(q1, alpha) | A -> alpha pertenece a P}
delta(q1, a, a)         = {(q1, epsilon)} para cada terminal a
delta(q1, epsilon, Z0) = {(q2, Z0)}
```

La pila se escribe con el tope a la izquierda. El estado q1 expande no terminales y consume terminales. El estado q2 es el único estado de aceptación.

La definición de PDA conserva la estructura de un autómata finito: estados, alfabeto de entrada, estado inicial, transiciones y estados de aceptación. Agrega el alfabeto de pila, el símbolo inicial de pila y las operaciones de push y pop. En la implementación, la pila se guarda como una tupla con el fondo a la izquierda y el tope a la derecha; al imprimir se invierte para mostrar el tope a la izquierda.

La transición por producciones hace que el PDA sea no determinista cuando un no terminal tiene varias producciones. La simulación explora las configuraciones (estado, entrada restante,

pila) y registra el camino que lleva a la aceptación.

Resumen de transiciones por producción:

```
delta(q1, epsilon, S)    -> q1 empujando S -> NP VP
delta(q1, epsilon, VP)   -> q1 empujando VP -> VP PP | V NP | cooks | drinks | eats | cuts
delta(q1, epsilon, PP)   -> q1 empujando PP -> P NP
delta(q1, epsilon, NP)   -> q1 empujando NP -> Det N | he | she
delta(q1, epsilon, V)    -> q1 empujando V -> cooks | drinks | eats | cuts
delta(q1, epsilon, P)    -> q1 empujando P -> in | with
delta(q1, epsilon, N)    -> q1 empujando N -> cat | dog | beer | cake | juice | meat | soup | fork
| knife | oven | spoon
delta(q1, epsilon, Det) -> q1 empujando Det -> a | the
delta(q0, epsilon, Z0)  -> q1 empujando S Z0
delta(q1, a, a)          -> q1 para cada terminal a
delta(q1, epsilon, Z0)  -> q2
```

La simulación usa una cola de configuraciones y un conjunto de configuraciones visitadas para evitar ciclos. También aplica una poda: si la pila tiene más símbolos que tokens restantes, la rama se descarta, porque cada no terminal de esta gramática deriva al menos un terminal.

3. PDA determinista LR/SLR

Además del PDA no determinista, se construye un PDA determinista usando una tabla SLR(1).

Primero se calculan los conjuntos FIRST y FOLLOW. Luego se construye la colección canónica de items LR(0) con las operaciones closure y goto. Con esos items se llena la tabla de acciones y la tabla de gotos.

La tabla obtenida tiene:

```
Estados LR(0): 32
Acciones: 196
Gotos: 12
Conflictos: 0
```

No hay conflictos shift/reduce ni reduce/reduce. Por eso la gramática es SLR(1) y el reconocimiento puede hacerse de forma determinista con una pila de estados.

El algoritmo de parseo usa:

- Una pila de estados.
- Una pila de valores semánticos para construir el árbol de parseo.
- La entrada terminada en $.

En cada paso consulta la acción correspondiente al estado del tope y al símbolo de entrada. Si la acción es shift, avanza la entrada y apila el nuevo estado. Si es reduce, reemplaza el lado derecho de una producción por su lado izquierdo y apila el estado de goto. Si es accept, la cadena se acepta.

Este PDA determinista reconoce el mismo lenguaje que el PDA no determinista, pero en tiempo lineal.

4. Algoritmo CYK

La tabla se define como tabla[i][j], donde la celda contiene los no terminales que derivan la subcadena tokens[i:j].

El algoritmo es:

1. Para cada token, agregar los no terminales A tales que A -> token. 2. Para cada longitud mayor que 1, probar todos los puntos de corte k. 3. Para cada producción A -> B C, agregar A si B está en tabla[i][k] y C está en tabla[k][j]. 4. Al final, la cadena se acepta si S esta en tabla[0][n].

Además de la tabla, se guardan retroapuntadores. Cada vez que se agrega un no terminal se registra el punto de corte y la producción usada. Con esos datos se reconstruye el árbol de parseo y la derivación por la izquierda.

5. Ejemplos y trazas

5.1 Cadena valida: "he cooks"

PDA no determinista:

```
(q0, he cooks, Z0)
|- (q1, he cooks, S Z0)           [iniciar con S]
|- (q1, he cooks, NP VP Z0)       [expandir S -> NP VP]
|- (q1, he cooks, he VP Z0)       [expandir NP -> he]
|- (q1, cooks, VP Z0)             [consumir 'he']
|- (q1, cooks, cooks Z0)          [expandir VP -> cooks]
|- (q1, epsilon, Z0)              [consumir 'cooks']
|- (q2, epsilon, Z0)              [aceptar]
Resultado: ACEPTADA
Operaciones: total=35, transiciones=14, pila=19, comparaciones=2, configuraciones=12
```

PDA determinista LR/SLR:

```
Pila: 0              Entrada: he cooks        Accion: shift 5
Pila: 0 5            Entrada: cooks           Accion: reduce NP -> he
Pila: 0 2            Entrada: cooks           Accion: shift 22
Pila: 0 2 22         Entrada: epsilon         Accion: reduce VP -> cooks
Pila: 0 2 21         Entrada: epsilon         Accion: reduce S -> NP VP
Pila: 0 3            Entrada: epsilon         Accion: accept
Resultado: ACEPTADA
Operaciones: total=6, shifts=2, reducciones=3
```

Árbol de parseo:

```
S
|-- NP
|   |-- he
|-- VP
    |-- cooks
```

CYK:

```
[0,1) 'he' += NP       (NP -> he)
[1,2) 'cooks' += VP    (VP -> cooks)
[1,2) 'cooks' += V     (V -> cooks)
[0,2) 'he cooks' += S (S -> NP VP; k=1, NP en [0,1) y VP en [1,2))

Tabla:
[0,1) 'he' -> {NP}
[1,2) 'cooks' -> {V, VP}
[0,2) 'he cooks' -> {S}

Derivación por la izquierda:
S => NP VP => he VP => he cooks

Operaciones: total=59, terminales=50, binarias=5, inserciones=4
```

5.2 Cadena invalida: "the cat eats cake"

Los tres métodos rechazan la cadena. La tabla CYK muestra que no se puede formar un NP para "cake", porque falta un determinante. La celda final no contiene S:

```
[0,1) 'the' -> {Det}
[1,2) 'cat' -> {N}
[2,3) 'eats' -> {V, VP}
[3,4) 'cake' -> {N}

[0,2) 'the cat' -> {NP}
[1,3) 'cat eats' -> {}
[2,4) 'eats cake' -> {}

[0,3) 'the cat eats' -> {S}
[1,4) 'cat eats cake' -> {}

[0,4) 'the cat eats cake' -> {}

Resultado: RECHAZADA
Operaciones CYK: total=157, terminales=100, binarias=50, inserciones=7
```

El PDA no determinista explora las ramas posibles y no alcanza q2 con la entrada consumida. El PDA LR/SLR detecta el error cuando la tabla no tiene una acción válida para el estado actual.

5.3 Resumen con el archivo de ejemplo

he cooks	ACEPTADA	ACEPTADA	ACEPTADA
she drinks the juice	ACEPTADA	ACEPTADA	ACEPTADA
the cat eats the cake	ACEPTADA	ACEPTADA	ACEPTADA
the dog drinks the beer in the oven	ACEPTADA	ACEPTADA	ACEPTADA
the cat cuts the meat with the knife	ACEPTADA	ACEPTADA	ACEPTADA
she eats the soup with the spoon in the oven	ACEPTADA	ACEPTADA	ACEPTADA
the cat cooks in the oven with the fork	ACEPTADA	ACEPTADA	ACEPTADA
the cat	RECHAZADA	RECHAZADA	RECHAZADA
eats the cake	RECHAZADA	RECHAZADA	RECHAZADA
the cat eats cake	RECHAZADA	RECHAZADA	RECHAZADA
he cooks the	RECHAZADA	RECHAZADA	RECHAZADA
the cat in the oven	RECHAZADA	RECHAZADA	RECHAZADA
cat eats the cake	RECHAZADA	RECHAZADA	RECHAZADA
the cat eats the cake in	RECHAZADA	RECHAZADA	RECHAZADA

En todos los casos los tres métodos coinciden.

6. Tasa de crecimiento

El análisis se hizo contando operaciones, no midiendo tiempo. La familia de cadenas usada fue:

```
he cooks
he cooks in the cat
he cooks in the cat in the cat
...
```

Cada cadena tiene longitud n = 2 + 3k tokens.

Operaciones contadas:

- CYK: comprobaciones de producciones terminales y binarias, más inserciones en la tabla.
- PDA no determinista: transiciones evaluadas, operaciones de pila y comparaciones de símbolos.
- PDA determinista: acciones de parseo, es decir, shifts, reducciones y aceptación.

Resultados seleccionados:

k	n	CYK	NPDA	DPDA	CYK/n^3
0	2	59	35	6	7.3750
5	17	4544	2445	51	0.9249
12	38	46733	11993	114	0.8517
20	62	200249	31725	186	0.8402
25	77	382484	48835	231	0.8378

Ajustes empíricos:

```
CYK:  operaciones aproximadamente 0.84 * n^3
      pendiente local log-log = 2.9889
NPDA: operaciones aproximadamente 9.03 * n^1.9778
      pendiente local log-log = 1.9915
DPDA: operaciones aproximadamente 3.00 * n
      pendiente local log-log = 1.0000
```

Análisis asintótico:

Metodo	Tiempo	Espacio
CYK	Theta(n^3)	Theta(n^2)
NPDA	peor caso exponencial	exponencial
DPDA	Theta(n)	Theta(n)

CYK es cubico porque recorre longitudes, posiciones iniciales, puntos de corte y producciones. Con una gramática fija, el número de producciones es constante y el costo dominante es n^3.

El PDA no determinista tiene peor caso exponencial en general, porque la simulación explora configuraciones y la pila puede crecer. En esta gramática, la poda por número de símbolos en la pila y la repetición de estructuras hace que el crecimiento empírico sea cuadrático. Aun así, su constante y su cantidad de operaciones son mayores que las del PDA determinista.

El PDA LR/SLR es lineal porque cada token produce un shift y cada producción reducida produce

una reducción. El número de reducciones es proporcional al número de nodos del árbol de parseo, que es O(n). Su costo empírico es aproximadamente 3n operaciones.

7. Comparación de desempeño

CYK es un método ascendente, sistemático y polinomial. Llena toda la tabla, por lo que su costo es cubico y su memoria cuadrática. Es apropiado para gramáticas en CNF y maneja ambigüedad sin cambiar el algoritmo.

El PDA no determinista es la conversión directa de la gramática. Su ventaja es que muestra el funcionamiento de la pila y las descripciones instantáneas. Su desventaja es que debe explorar varias ramas y, en el peor caso, puede tener crecimiento exponencial. En esta gramática el crecimiento observado es cuadrático, pero sigue siendo más costoso que el PDA determinista.

El PDA determinista LR/SLR es el más eficiente de los tres. Reconoce en tiempo lineal y construye el árbol de parseo directamente. Su limitación es que necesita que la gramática permita una tabla determinista sin conflictos. En este caso la gramática es SLR(1), por lo que el método se puede aplicar.

Comparación para cadenas de la familia analizada:

- DPDA: alrededor de 3n operaciones.
- NPDA: alrededor de 9.03 n^1.98 operaciones.
- CYK: alrededor de 0.84 n^3 operaciones.

Para cadenas cortas, las diferencias son pequeñas. Para cadenas largas, el crecimiento cubico de CYK y el crecimiento cuadrático del NPDA hacen que el DPDA sea claramente más eficiente.

8. Conclusiones

- La conversión CFG -> PDA produce un reconocedor no determinista correcto y permite mostrar descripciones instantáneas.
- La gramática admite una tabla SLR(1) sin conflictos, lo que permite construir un PDA determinista lineal.
- Los tres métodos coinciden en todas las cadenas validas e invalidas probadas.
- CYK tiene tiempo Theta(n^3) y espacio Theta(n^2).
- El PDA determinista tiene tiempo Theta(n) y espacio Theta(n).
- El PDA no determinista tiene peor caso exponencial, aunque en esta gramática el crecimiento empírico observado es cuadrático.

9. Referencias

- Hopcroft, J. E., Motwani, R., Ullman, J. D. Introduction to Automata Theory, Languages, and Computation.
- Sipser, M. Introduction to the Theory of Computation.
- Aho, A. V., Lam, M. S., Sethi, R., Ullman, J. D. Compilers: Principles, Techniques, and Tools.